

Introduction to Object Oriented Programming (OOP)

1. Competency

After taking this module, students may be able to:

1. Recognize the difference between object oriented and procedural paradigm in programming.
2. Recognize the basic concept of OOP
3. Implement OOP in java programming

2. Introduction

2.1 Procedural Programming vs Object Oriented Programming

In procedural programming, a program is divided into some sub-program called function. It is difference with OOP, the main concept is a program is divided into object where object consists of state and method.

OOP is more flexible and modular, it can be described that each part of program won't be disturbed if a feature is updated. It is difference with procedural programming where the program may change overall when an update is implemented.

More clearly, this is the sample of procedural programming and object-oriented programming.

- **Procedural Programming**

J Procedural.java > ...

```
1  public class Procedural {  
    Run | Debug  
2      public static void main(String[] args) {  
3          String itemName1 = "Coffee";  
4          float price1 = 120000.0f;  
5          int quantity1 = 3;  
6  
7          String itemName2 = "Sugar";  
8          float price2 = 30000.0f;  
9          int quantity2 = 1;  
10  
11         System.out.println(itemName1 + ": " + total(price1, quantity1));  
12         System.out.println(itemName2 + ": " + total(price2, quantity2));  
13     }  
14  
15     static float total(float price, int quantity) {  
16         return price * quantity;  
17     }  
18 }
```

- **Object Oriented Programming**

J OOP.java > ...

```
1  class Product {
2
3      private final String itemName;
4      private final float price;
5
6      public Product (String itemName, float price) {
7          this.itemName = itemName;
8          this.price = price;
9      }
10
11     public float total(int quantity) {
12         return price * quantity;
13     }
14
15     public String getName() {
16         return itemName;
17     }
18 }
19
20 public class OOP {
21     Run | Debug
22     public static void main(String[] args) {
23
24         Product product1 = new Product("Coffee", 120000.0f);
25         Product product2 = new Product("Sugar", 30000.0f);
26
27         System.out.println(product1.getName() + ": " + product1.total(3));
28         System.out.println(product2.getName() + ": " + product2.total(1));
29     }
30 }
```

Based on these two code snippets, object-oriented code is more organized and efficient than procedural code because it avoids repetitive code. This can be seen in the example of object-oriented code. It is clear that a new object is created using the `new` keyword followed by the class name.

NB:

In this module, we will try to create class/classes, creating object, and calling methods in object. More explanation about class anatomy will be discussed in the next module.

2.2 Basic Concept of OOP

Some aspects are known in OOP, such:

a. Object

Object is a sequence of program which consists of **state** and **behavior**. Object in software is modelled to be similar of object in real world. State in object is attribute of the object itself, in example: object Sepeda has state like brand, speed, and gear. Behavior is a something that the object can do, in example: in object Bike, the behaviors are: speedAcceleration, gearChange, and speedDeceleration.

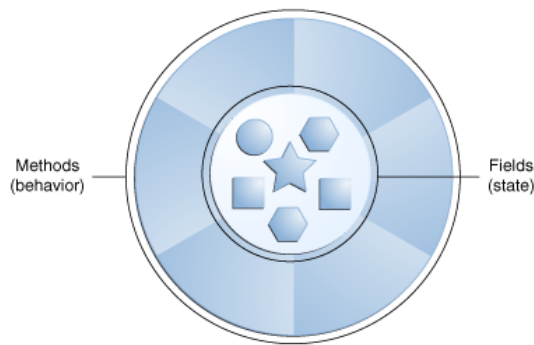


Figure 1. Illustration of object in software

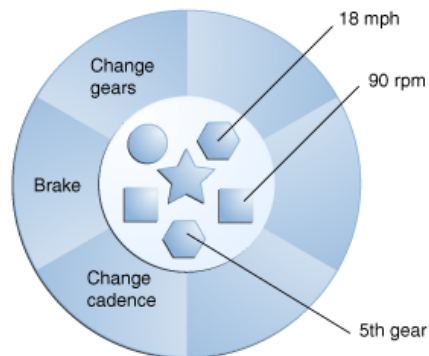


Figure 2. Illustration of state and behavior in object Bike

b. Class

Class is a blueprint or prototype from object. Taking example from object Bike, although there are many kinds of bike based on brand and model, the blueprint of bike will be the same such component and characteristic. The one you may belong in your home can be defined as instantiation from class Bike.

c. Encapsulation

Also known as **information-hiding**. The complex scheme in program may not be shown necessarily. In bike or bicycle, we can change gear position by pressing gear knob at the handlebar, we may need not to know how the scheme in changing gear works technically.

d. Inheritance

Inheritance allow us to organize the structure of program naturally. We can expand the functionality of program without make a large modification in that program. In example, bicycle can be inherited to another model like mountain bike and road bike where each model has additional component which is not owned by common bike. In another word, mountain bike and road bike inherit bicycle.

e. Polymorphism

Polymorphism imitates real world object characteristics where an object may has different form. In example, plane object can be inherited to be jet plane and helicopter which have ability to increasing the speed. But, the method to increase the speed is different between those models because of different engine used technically.

3. Experiment Activity

3.1 Experiment Activity 1

In this activity, we would demonstrate about how creating class, creating object, then accessing method in the class.

1. Open NetBeans, create project **BikeDemo**.
2. Create class Sepeda. Right click on the package **BikeDemo** – New – Java Class.
3. Type this code in class Bike.

```
1  public class Bike{
2      private String brand;
3      private int speed;
4      private int gear=1;
5      // Gear 1: max 5 km/h, Gear 2: max 10 km/h, ... Gear 6: max 60 km/h
6      private final int[] GEAR_SPEED_LIMITS = {5, 10, 25, 30, 40, 60};
7      public void setBrand(String brandName){
8          brand = brandName;
9      }
10     public void gearChanges(int gearValue){
11         if (gearValue < 1 || gearValue > 6) {
12             System.out.println("Invalid gear value. Gear must be between 1 and 6.");
13         }
14         else{
15             gear = gearValue;
16         }
17     }
18     public int speedAcceleration(int increment) {
19         speed += increment;
20         if (speed > GEAR_SPEED_LIMITS[gear - 1]) {
21             speed = GEAR_SPEED_LIMITS[gear - 1];
22         }
23         return speed;
24     }
25     public int speedDeceleration(int decrement) {
26         speed -= decrement;
27         if (speed < 0) {
28             speed = 0;
29         }
30         return speed;
31     }
32     public void printInfo(){
33         System.out.println("Brand : " + brand);
34         System.out.println("Speed : " + speed);
35         System.out.println("Gear : " + gear);
36     }
37 }
```

4. Then in the class main, type this code.

```

1  public class BikeDemo{
    Run | Debug
2      public static void main(String[] args){
3          Bike mountainBike1 = new Bike();
4          Bike mountainBike2 = new Bike();
5
6          mountainBike1.setBrand("Trek");
7          mountainBike1.speedAcceleration(10);
8          mountainBike1.gearChanges(2);
9          mountainBike1.printInfo();
10
11         mountainBike2.setBrand("Giant");
12         mountainBike2.speedAcceleration(20);
13         mountainBike2.gearChanges(3);
14         mountainBike2.printInfo();
15     }
16 }

```

5. Check the result, it should be like this:

```

Brand : Trek
Speed : 5
Gear : 2
Brand : Giant
Speed : 5
Gear : 3

```

3.2 Experiment 2

In this activity, we would demonstrate inheritance. We perform this inheritance technique by creating class **RoadBike** which is inherited from class **Bike**. Basically, class **RoadBike** is similar with class **Bike** but it has tire width state. By doing this, we may not create class **RoadBike** from the scratch, it just inherits from class **Bike** to class **RoadBike**.

More detail explanation about inheritance will be discussed in the next module.

1. Still in the project **BikeDemo**. Create a class named **RoadBike**.
2. Add a code **extends Bike** at the class **RoadBike** declaration. This extends points that class **RoadBike** inherits class **Bike**.

```

1  public class RoadBike extends Bike {
2
3  }

```

3. Complete the class **RoadBike** by typing this code:

```

1  public class RoadBike extends Bike {
2      private int tireWidth;
3
4      public void setTireWidth(int width) {
5          tireWidth = width;
6      }
7
8      @Override
9      public void printInfo() {
10         super.printInfo();
11         System.out.println("Tire Width : " + tireWidth + " mm");
12         System.out.println("Bike Type : Road Bike");
13     }
14 }

```

4. Next, in the main class, add this code.

```

1  public class BikeDemo{
2      Run | Debug
3      public static void main(String[] args){
4          Bike mountainBike1 = new Bike();
5          Bike mountainBike2 = new Bike();
6          RoadBike roadBike1 = new RoadBike();
7
8          mountainBike1.setBrand("Trek");
9          mountainBike1.speedAcceleration(10);
10         mountainBike1.gearChanges(2);
11         mountainBike1.printInfo();
12
13         mountainBike2.setBrand("Giant");
14         mountainBike2.speedAcceleration(20);
15         mountainBike2.gearChanges(3);
16         mountainBike2.printInfo();
17
18         roadBike1.setBrand("Specialized");
19         roadBike1.setTireWidth(25);
20         roadBike1.speedAcceleration(15);
21         roadBike1.gearChanges(4);
22         roadBike1.printInfo();
23     }
24 }

```

5. Check the result:

```
Brand : Trek  
Speed : 5  
Gear : 2  
Brand : Giant  
Speed : 5  
Gear : 3  
Brand : Specialized  
Speed : 5  
Gear : 4  
Tire Width : 25 mm  
Bike Type : Road Bike
```

4. Conclusion

Based on the activity, we have demonstrated about what is paradigm of OOP and the implementation into a simple program. Inheritance also one of the importance features in OOP, we have demonstrated it by implementing class **RoadBike**.

We also know that class SepedaGunung is similar with class **Bike** (having gear and speed state, also some behaviors like increasing speed, braking, changing gear) but with additional state named **tire width**. So we may not necessarily to create class RoadBike from the scratch, we use **extends** or just inherits from class Bike, then we add a new feature which is not owned by class Bike before. This is the characteristic of OOP which is not having in procedural programming.

5. Question Test

1. Explain the difference between object and class!
2. State your reason why gear and brand can be classified as attribute for Bike object!
3. State one of OOP better point than procedural programming!
4. Is it allowed to define two attributes in one line code such "**public String name, address;**"?
5. In RoadBike class, state your reason why brand, speed, and gear attributes are not written again in this class!

6. Assignment

1. Follow these instructions to make your practical assignment is performed systematically:
 - a. Take 4 photographs of objects around you, 2 objects must be implementation of inheritance concept, example: refrigerator, chair, living room table, desk! As we know that living room table and desk are inherited by table class.
 - b. Observe those objects to define the attribute and method!
 - c. Convert those objects into four classes in Java programming!
 - d. Add one additional class as a class which inherits its attribute and method to living room table class and desk class!
 - e. Add two attributes for each class!
 - f. Add three methods for each class including a method for showing the information!
 - g. Add one class named Demo for main class!
 - h. Instance an object for each class!

- i. Apply each method for each object in main class!
- j. The example which is mentioned in point 1.a should not be included in your task!